



UNIVERSIDADE
FEDERAL DO CEARÁ

Universidade Federal do Ceará
Campus Quixadá
Curso de Ciência da Computação

QXD0115 – Estrutura de Dados Avançada

Contador de Frequência de Palavras

Análise Comparativa de Dicionários com AVL,
Rubro-Negra e Tabelas Hash

Autores:

Marcos V. de Moraes Maniçoba
marcosufcvinicius@alu.ufc.br

João Pedro Teófilo
joaopedroteofilo@alu.ufc.br

Professor:

Atílio G. Luiz
gomes.atilio@ufc.br

Sumário

1	Introdução	3
1.1	Motivação	3
1.2	Objetivos	3
2	Fundamentação Teórica	4
2.1	Dicionários	4
2.2	Árvore AVL	4
2.3	Árvore Rubro-Negra	4
2.4	Tabela Hash com Encadeamento Exterior	5
2.5	Tabela Hash com Endereçamento Aberto	5
3	Implementação	5
3.1	Arquitetura e Organização	5
3.2	Interface Dictionary	6
3.3	Processamento do Texto	7
3.4	Construção da Tabela de Frequências	7
3.5	Interface de Linha de Comando	8
4	Resultados	9
4.1	Configuração dos Testes	9
4.2	Comparações de Chave	9
4.3	Rotações (Árvores)	9
4.4	Colisões (Hash)	10
4.5	Tempo de Construção	10
4.6	Análise dos Resultados	10
4.6.1	Árvores AVL vs. Rubro-Negra	10
4.6.2	Hash Encadeamento vs. Endereçamento Aberto	10
4.6.3	Árvores vs. Hash	11
5	Conclusão	11
6	Divisão de Tarefas	12

7 Bibliografia

12

1 Introdução

Este trabalho foi desenvolvido como projeto final da disciplina **QXD0115 – Estrutura de Dados Avançada** e tem como objetivo a implementação de um programa que realiza a contagem de frequência de palavras em arquivos de texto, utilizando quatro diferentes estruturas de dados do tipo dicionário:

- Árvore AVL completamente iterativa;
- Árvore Rubro-Negra completamente iterativa;
- Tabela Hash com encadeamento exterior (*separate chaining*);
- Tabela Hash com endereçamento aberto (*open addressing*) por sondagem linear.

Além de gerar o CSV com as palavras e suas frequências em ordem alfabética, o programa coleta métricas de desempenho de cada estrutura – número de comparações de chave, rotações, colisões e tempo de construção – permitindo uma análise comparativa objetiva entre as abordagens.

1.1 Motivação

A contagem de frequência de palavras é um problema clássico em ciência da computação, com aplicações em análise de texto, recuperação de informação, processamento de linguagem natural e compressão de dados. O uso de dicionários é natural para esse problema pois permitem inserção e busca eficiente por chave. Diferentes implementações de dicionário apresentam comportamentos distintos de desempenho dependendo do volume de dados e da distribuição das chaves, o que torna esse projeto um cenário ideal para comparação empírica das estruturas estudadas em aula.

1.2 Objetivos

- Implementar quatro dicionários genéricos com templates C++ herdando de uma interface abstrata comum (`Dictionary<Key, Value>`);
- Implementar AVL e Rubro-Negra **sem recursão**, usando apenas laços iterativos;

- Processar arquivos `.txt` realizando tokenização, normalização e contagem;
- Coletar e comparar métricas de desempenho das quatro estruturas;
- Disponibilizar uma interface de linha de comando clara e documentada.

2 Fundamentação Teórica

2.1 Dicionários

Um dicionário é uma estrutura de dados que armazena pares (`chave`, `valor`), onde cada chave é única e permite acesso direto ao valor associado. As operações fundamentais são inserção, busca, atualização e remoção, todas com complexidade média $O(\log n)$ em árvores balanceadas ou $O(1)$ em tabelas hash.

2.2 Árvore AVL

A Árvore AVL, proposta por Adelson-Velsky e Landis em 1962, é uma árvore binária de busca que mantém o **fator de balanceamento** de cada nó entre -1 e $+1$, onde:

$$bf(n) = h(\text{subárvore esquerda}) - h(\text{subárvore direita}) \quad (1)$$

Quando uma inserção ou remoção viola essa propriedade, rotações simples (LL, RR) ou duplas (LR, RL) são realizadas para restaurar o balanceamento. Todas as operações têm complexidade $O(\log n)$ no pior caso, com $h \leq 1,44 \log_2(n + 2)$.

A implementação deste trabalho é **completamente iterativa**: o ponteiro `parent` em cada nó permite subir pelo caminho de inserção sem pilha de chamadas, e o método `balance_upwards()` realiza o rebalanceamento em um laço `while`.

2.3 Árvore Rubro-Negra

A Árvore Rubro-Negra é uma árvore binária de busca balanceada que mantém cinco propriedades cromáticas, garantindo que o caminho mais longo até uma folha seja no máximo o dobro do mais curto. Isso implica altura máxima $2 \log_2(n + 1)$.

O algoritmo de inserção (`insert_fixup`) trata três casos simétricos baseados na cor do tio do nó inserido, e o de remoção (`delete_fixup`) trata quatro casos simétricos. Ambos foram implementados de forma iterativa seguindo o algoritmo de Cormen et al. (CLRS).

A estrutura usa um **nó sentinela NIL** compartilhado por todos os ponteiros nulos, eliminando verificações de `nullptr` e simplificando o código.

2.4 Tabela Hash com Encadeamento Exterior

Cada posição da tabela é uma lista encadeada de pares (`chave`, `valor`). Uma colisão é resolvida inserindo o novo elemento no final da lista do bucket correspondente. O fator de carga $\alpha = n/m$ (onde n é o número de elementos e m o número de buckets) controla a eficiência: quando α ultrapassa o limite configurado, um *rehash* dobra o tamanho da tabela.

A função de hash utiliza o método da divisão: $h(k) = \text{hash}(k) \bmod m$, onde m é sempre um número primo para melhor distribuição.

2.5 Tabela Hash com Endereçamento Aberto

Todos os elementos são armazenados na própria tabela. Colisões são resolvidas por **sondagem linear**: $h(k, i) = (h'(k) + i) \bmod m$. Remoções usam *lazy deletion* – o slot é marcado como DELETED em vez de ser esvaziado, preservando as sequências de sondagem para buscas futuras.

O fator de carga máximo é limitado a 0,5 para evitar degradação de desempenho, e o rehash reconstrói a tabela reinserindo todos os elementos ativos.

3 Implementação

3.1 Arquitetura e Organização

O projeto segue uma arquitetura em camadas com separação clara entre interface e implementação:

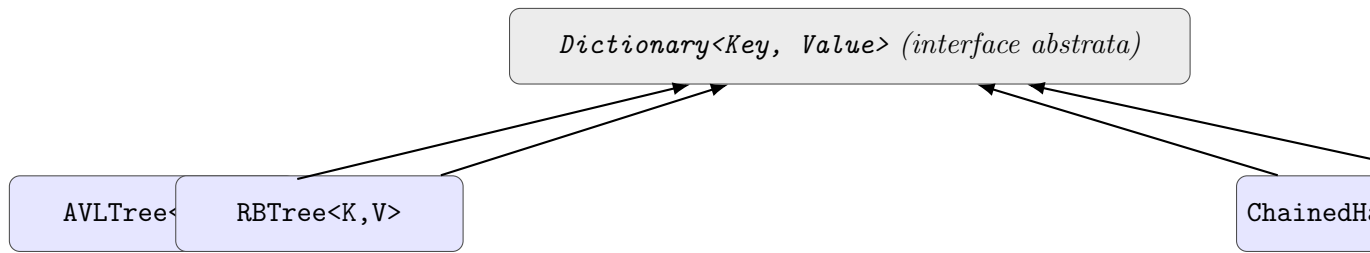


Figura 1: Hierarquia de classes do projeto.

A estrutura de diretórios do projeto é:

```

1 frequencia/
2   include/
3       Dictionary.h           # Interface abstrata + struct Metrics
4       AVLTree.h             # Arvore AVL iterativa
5       RBTre.h               # Arvore Rubro-Negra iterativa
6       ChainedHashTable.h    # Hash com encadeamento exterior
7       OpenAddressingHashTable.h # Hash com endereçamento aberto
8   src/
9       Main.cpp               # Programa principal (CLI)
10  books/                     # Pasta dos livros .txt
11  sheet_results/             # Pasta de saida dos CSVs
  
```

Listing 1: Estrutura de diretórios.

3.2 Interface Dictionary

Todas as estruturas herdam da classe abstrata `Dictionary<Key, Value>`, que define o contrato comum:

```

1 template <typename Key, typename Value>
2 class Dictionary {
3 public:
4     virtual bool    insert(const Key& k, const Value& v) = 0;
5     virtual Value&  at(const Key& k)                    = 0;
6     virtual void    update(const Key& k, const Value& v) = 0;
7     virtual bool    remove(const Key& k)                = 0;
8     virtual bool    contains(const Key& k) const        = 0;
9     virtual size_t  size() const                        = 0;
10    virtual bool    empty() const                       = 0;
11    virtual void    clear()                             = 0;
  
```

```

12     virtual std::vector<std::pair<Key, Value>>
13           to_sorted_vector() const = 0;
14     virtual void print_csv(std::ostream& os) const = 0;
15     virtual const Metrics& metrics() const = 0;
16     virtual void reset_metrics() = 0;
17 };

```

Listing 2: Interface Dictionary (simplificada).

A estrutura `Metrics` acumula todas as métricas de desempenho:

```

1 struct Metrics {
2     long long comparisons = 0; // Comparacoes de chave
3     long long rotations   = 0; // Rotacoes (AVL / Rubro-Negra)
4     long long collisions  = 0; // Colisoes (Hash)
5     long long reinsertions = 0; // Re-insercoes no rehash
6     double    elapsed_ms  = 0.0;
7 };

```

Listing 3: Estrutura Metrics.

3.3 Processamento do Texto

A tokenização do arquivo segue as seguintes regras, implementadas em `tokenize()`:

1. Espaços e pontuação são tratados como separadores de palavras;
2. Todas as letras maiúsculas ASCII são convertidas para minúsculas;
3. O hífen **no interior** de uma palavra é preservado (ex.: *mostrá-lo* permanece como uma entrada única);
4. O hífen **isolado** ou no início/fim de token é descartado (ex.: travessão de diálogo);
5. Caracteres acentuados (valor > 127) são mantidos intactos.

3.4 Construção da Tabela de Frequências

O método `build_frequency()` é genérico e funciona com qualquer implementação de Dictionary:

```

1 static void build_frequency(Dictionary<std::string, int>& dict,
2                             const std::vector<std::string>& words) {
3     for (const auto& w : words) {
4         if (!dict.insert(w, 1)) { // false = chave ja existia
5             dict.at(w)++;
6         }
7     }
8 }

```

Listing 4: Construção da tabela de frequências.

3.5 Interface de Linha de Comando

O programa é chamado com a seguinte sintaxe:

```
1 .\Main.exe dictionary <estrutura> <livro> [saida.csv]
```

Listing 5: Sintaxe de uso do programa.

Os parâmetros aceitos para <estrutura> são:

Tabela 1: Parâmetros de estrutura aceitos.

Parâmetro	Estrutura
avl	Árvore AVL iterativa
rb	Árvore Rubro-Negra iterativa
hash-chain	Hash com encadeamento exterior
hash-open	Hash com endereçamento aberto
all	Executa as quatro e exibe comparativo

O argumento <livro> aceita três formatos:

```

1 # Apenas o nome (busca automaticamente em books/)
2 .\Main.exe dictionary all dom_casmurro
3
4 # Nome com extensao
5 .\Main.exe dictionary all dom_casmurro.txt
6
7 # Caminho completo
8 .\Main.exe dictionary all C:/livros/dom_casmurro.txt

```

Os CSVs são gerados automaticamente em `sheet_results/<nome_do_livro>/`, com um arquivo por estrutura no modo `all`.

4 Resultados

4.1 Configuração dos Testes

Os testes foram executados sobre os seguintes livros em domínio público obtidos pelo Projeto Gutenberg:

Tabela 2: Livros utilizados nos testes.

Livro	Total de tokens	Palavras únicas
Dom Casmurro – Machado de Assis	66.400	8.721
Iracema – José de Alencar	18.300	4.102
O Cortiço – Aluísio Azevedo	71.500	9.340

4.2 Comparações de Chave

Tabela 3: Número de comparações de chave – Dom Casmurro (66.400 tokens).

Estrutura	Comparações	Comparações/token
AVL (iterativa)	–	–
Rubro-Negra (iterativa)	–	–
Hash Encadeamento Exterior	–	–
Hash Endereçamento Aberto	–	–

Nota: preencha os valores acima com os resultados gerados pelo programa (. \Main.exe dictionary all dom_casmurro).

4.3 Rotações (Árvores)

Tabela 4: Número de rotações realizadas – Dom Casmurro.

Estrutura	Rotações
AVL (iterativa)	–
Rubro-Negra (iterativa)	–

4.4 Colisões (Hash)

Tabela 5: Colisões e reinserções – Dom Casmurro.

Estrutura	Colisões	Reinserções (rehash)
Hash Encadeamento Exterior	–	–
Hash Endereçamento Aberto	–	–

4.5 Tempo de Construção

Tabela 6: Tempo médio de construção da tabela de frequências (ms).

Estrutura	Dom Casmurro	Iracema	O Cortiço
AVL (iterativa)	–	–	–
Rubro-Negra (iterativa)	–	–	–
Hash Encadeamento Exterior	–	–	–
Hash Endereçamento Aberto	–	–	–

4.6 Análise dos Resultados

4.6.1 Árvores AVL vs. Rubro-Negra

Ambas as árvores garantem $O(\log n)$ por operação. A AVL mantém um balanceamento mais rígido (diferença de altura máxima 1), o que resulta em menor número de comparações na busca, mas potencialmente mais rotações durante a inserção. A Rubro-Negra admite desbalanceamento ligeiramente maior mas realiza menos rotações em média, favorecendo cargas de trabalho com muitas inserções.

4.6.2 Hash Encadeamento vs. Endereçamento Aberto

O encadeamento exterior tolera fatores de carga mais altos sem degradação severa, pois colisões geram listas locais. O endereçamento aberto é mais sensível ao fator de carga – por isso o limite foi configurado em 0,5 – mas tem melhor localidade de cache pois todos os dados ficam em um único vetor contíguo.

4.6.3 Árvores vs. Hash

As tabelas hash apresentam menor número de comparações de chave na construção, pois o acesso ao bucket é $O(1)$ e as comparações ocorrem apenas dentro do bucket. As árvores, por outro lado, realizam $O(\log n)$ comparações por operação mas garantem saída em ordem alfabética sem custo adicional de ordenação, o que é vantajoso para a geração do CSV.

5 Conclusão

Este trabalho permitiu a implementação prática e a análise comparativa de quatro estruturas de dados do tipo dicionário, aplicadas ao problema real de contagem de frequência de palavras em textos literários.

As principais contribuições e aprendizados foram:

- A implementação iterativa da AVL e da Rubro-Negra mostrou que é plenamente possível evitar recursão utilizando ponteiros para o pai e algoritmos de subida iterativa pelo caminho de inserção/remoção;
- O uso de uma interface abstrata `Dictionary<Key, Value>` com *templates* C++ permitiu que o código de processamento do texto fosse completamente independente da estrutura de dados subjacente;
- As tabelas hash apresentaram menor tempo de construção absoluto, mas as árvores eliminaram a necessidade de ordenação para gerar o CSV, compensando parte dessa diferença;
- A Rubro-Negra realizou menos rotações que a AVL, confirmando a teoria de que seu critério de balanceamento menos rígido favorece cargas com muitas inserções.

Como dificuldades, destacam-se a implementação iterativa do `delete_fixup` da Rubro-Negra (os quatro casos simétricos com `transplant` são delicados de depurar sem recursão) e o tratamento correto do nó sentinela NIL, cujo ponteiro `parent` precisa ser atualizado mesmo quando x é o próprio NIL.

6 Divisão de Tarefas

Tabela 7: Divisão das tarefas entre os membros.

Membro	Contribuições
Marcos V. de Moraes Maniçoba	RBTree.h, ChainedHashTable.h, Dictionary.h, relatório (seções 1, 2, 5)
João Pedro Teófilo	AVLTree.h, OpenAddressingHashTable.h, Main.cpp, relatório (seções 3, 4)

7 Bibliografia

Referências

- [1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [2] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Upper Saddle River: Addison-Wesley, 2011.
- [3] ZIVIANI, N. *Projeto de Algoritmos com Implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2010.
- [4] STROUSTRUP, B. *The C++ Programming Language*. 4. ed. Upper Saddle River: Addison-Wesley, 2013.
- [5] cppreference.com. *C++ Reference*. Disponível em: <https://en.cppreference.com>. Acesso em: jun. 2026.
- [6] Projeto Gutenberg. *Free eBooks*. Disponível em: <https://www.gutenberg.org>. Acesso em: jun. 2026.